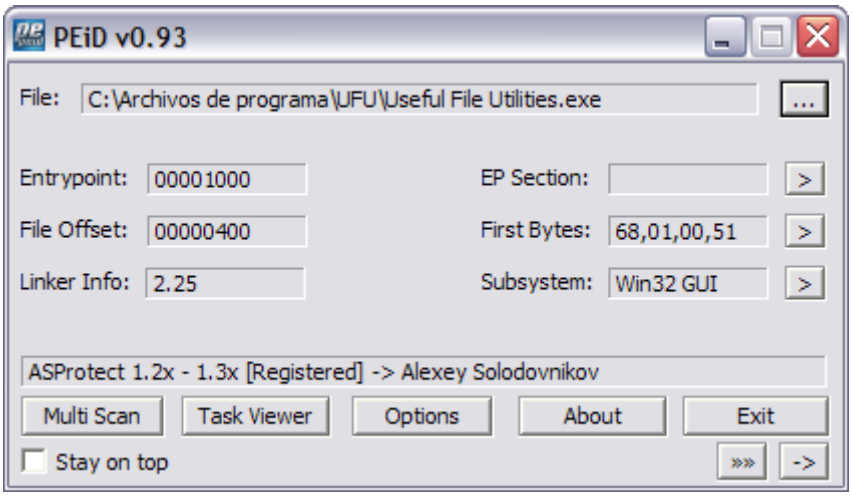
Primer acercamiento al Stolen Code parte 1- [por SyXe'05]

Programa:	Useful File Utilities v2.3.5
Protección:	ASProtect v1.2x - 1.3x [Registered] + Opciones varias
Objetivos:	Desempaketar el archivo ejecutable.
URL descarga:	http://DondeSea.com
Dificultad:	Fácil/Media
Herramientas:	OllyDbg v1.10 + PEiD v0.93 + ImpREC v1.6 + ConTEXT v0.98 + Hex WorkShop 4.23 + PEditor v1.7 + LordPE Deluxe + ToPo v1.2
Cracker:	SyXe'05 .::[CracksLatinoS]::. Fecha: 20-08-2006

Un saludo a todo CracksLatinoS!

Pues eso, un saludo a todos, hace algún tiempito que no escribo nada y me he decidido hoy, vamos a ver una forma útil de cómo desempakar un ASProtect de estos que tienen Stolen Code, en un principio puede ser un rival de entidad pero con un poco de esfuerzo se puede superar, no sin dificultades claro. No pretendo el camino único de hacer esto sino 'una forma más' de hacerlo o al menos la que yo utilicé, antes de empezar veamos nuestra víctima más de cerca con PEiD v0.93:



Nos dice que es un ASProtect 1.2x-1.3x [Registered], he de decir que dentro de esta versión he encontrado varios tipos de protección, entre ellas la más simple una redirección de iat y la más compleja una redirección de iat con destrucción de la tabla de saltos [JMP Table] y aparición de Stolen Code, además de insertar basura en la iat, veamos de cuál se trata en esta ocasión.

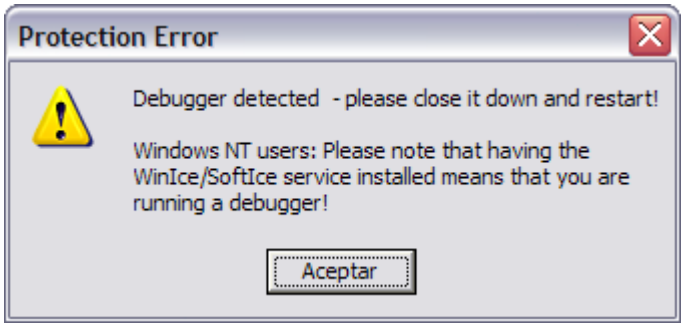
Antes estaría bien saber qué es eso del Stolen Code [***SCode*** de aquí en adelante]. Bien, un programa normalmente comienza con estas instrucciones:

```
PUSH EBP                    PUSH 0
MOV EBP, ESP                CALL DWORD PTR [GetModuleHandleA] / [InitCommonControls]
ADD ESP, -x                 ...etc...
...etc...
```

En versiones anteriores este packer eliminaba estas líneas y dejaba allí una serie de ceros que ejecutaba dentro del packer antes de saltar a la sección de código, ahora hace esto mismo pero en grandes porciones de código que van más allá de unas cuantas líneas, para asegurarme empaké un crackme con la opción Stolen Code activada y eliminó más de 1000h bytes del ejecutable! y eso que el crackme era realmente pequeño, consiste en ejecutar gran cantidad de código fuera del EXE protegido, lo que hace más difícil su reconstrucción. Por motivos personales hace tiempo que no me conecto a Internet y que no aparezco por la lista [esto cambiará el mes de Septiembre] que volveré ;) y no sé cómo está el tema de esta protección y como le he estado echando un vistazo desde un tiempito atrás he decidido escribir un tute, fue Juan José quién me pasó un programa con esta protección [Hat Trick Manager], le estuve dando vueltas sin lograr nada, y si tengo tiempo lo intentaré de nuevo, también un e-mail de ThunderPwr [al que no respondí, lo siento no domino el inglés ;(] en el que me comentaba algo acerca de esta misma protección. Creo que Natxo escribió un tute acerca del desempakado pero sólo tengo la primera parte donde aún no dice nada del SCode [¿escribió la segunda parte?], en fin.. que todo esto me empujó a estudiarlo por mí mismo y sacar mis propias conclusiones, así que sin perder más tiempo entraremos en materia ;)

Se levanta el telón...

Lo primero que haremos será abrir el EXE protegido en Olly y apretar F9 así sin más [con las excepciones deshabilitadas] para ver si corre, lo probamos en un Olly sin plugins y no corre, el tema está en el byte de IsDebuggerPresent que encontramos en [ebx+2] desde el EntryPoint --> [7FFDF002] en mi máquina:

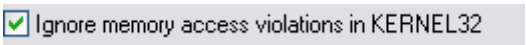


Es muy fácil superar esto, bien poniendo el byte directamente a cero o bien con el plugin adecuado. Si lo ponemos a cero el programa corre bien, su EP es éste:

Address	Hex dump	Disassembly		Comment
00401000	68 01005100	push	00510001	
00401005	E8 01000000	call	0040100B	
0040100A	C3	ret		
0040100B	C3	ret		
0040100C	D3F5	sal	ebp,cl	
0040100E	C8 9A9842	enter	989A,42	
00401012	194C6C A2	sbb	[esp+ebp*2-5E],ecx	
00401016	E4 E6	in	al,0E6	I/O command
00401018	92	xchg	eax,edx	
00401019	59	pop	ecx	
0040101A	- E0 81	loopdne	short 00400F9D	
0040101C	92	xchg	eax,edx	

Hay dos ejecutables, uno llamado "BatchReplacer.exe" que abre una consola de comandos tipo MS-DOS compilado en Delphi y otro llamado "Useful File Utilities.exe" que es el que tiene la protección, además hay un par de archivos *.plgn y uno de estos concretamente "BatchReplacer.plgn" también está empakado [según PEiD con ASPack v2.12] pero no me creo nada ya que le he echado un vistazo y parece también un ASProtect2.

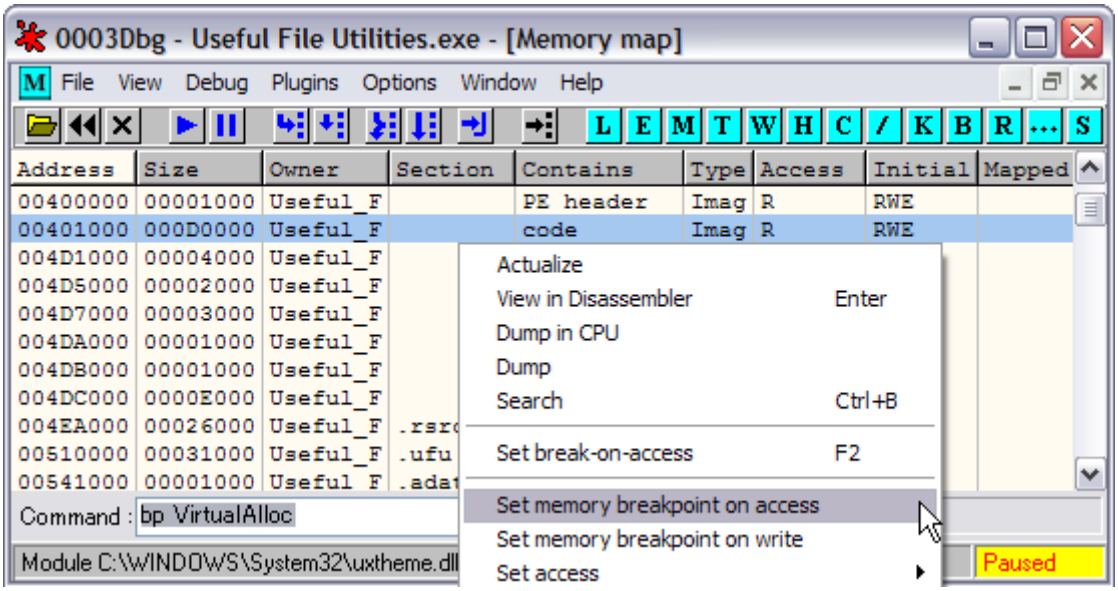
Ahora que conocemos la estructura del programa y los archivos que éste contiene vamos por faena, pulsamos ALT+O y en el cuadro EXCEPTIONS desmarcamos todas las casillas menos ésta:



Le dejamos marcada para que Olly ignore las excepciones que ocurran en la librería kernel32, y vamos pulsando {[F9] + [CTRL+F9]} para pasar las excepciones que vayan saltando hasta llegar a la última antes de que arranque el programa:

```
Address   Hex dump     Disassembly                          Comment
012AFB34  C601 4E      mov     byte ptr [ecx],4E
012AFB37  A4           movsb
012AFB38  D5 94        aad     94
012AFB3A  9F           lahf
012AFB3B  07           pop     es                           Modification of segment
012AFB3C  104B 59      adc     [ebx+59],cl
012AFB3F  BB C9693B67  mov     ebx,673B69C9
012AFB44  64:8F06      pop     dword ptr fs:[esi]
012AFB47  0000         add     [eax],al
012AFB49  EB 02        jmp     short 012AFB4D
```

Si la pasamos el programa corre así que vamos a la ventana de memoria [***Memory Map***] y ponemos un BPM ON ACCESS que abarque toda la sección de código:



Pasamos la excepción pulsando CTRL+F9 y para en una excepción distinta a la anterior que antes no salía, es ésta:

```
Address   Hex dump     Disassembly                          Comment
012AFDC3  0156 00      add     [esi],edx                    // Otra excepcion!
012AFDC6  842B         test    [ebx],ch
012AFDC8  C3           ret
012AFDC9  89C9         mov     ecx,ecx
012AFDCB  99           cdq
012AFDCC  D85448 F9    fcom    dword ptr [eax+ecx*2-7]
012AFDD0  77 02        ja      short 012AFDD4
```

En caso de no poner el BPM ésta no aparece, no tengo ninguna explicación aparente que explique este comportamiento, nosotros la pasamos igualmente con SHIFT+F9 y Olly para en lo que debería ser el OEP:

```
Address   Hex dump     Disassembly                          Comment
00406C60  E8 9B935B01  call    019C0000
00406C65  3C 8B        cmp     al,8B
00406C67  C0E8 93      shr     al,93                        Shift constant out of range 1..31
00406C6A  93           xchg    eax,ebx
00406C6B  5D           pop     ebp
00406C6C  0169 8B      add     [ecx-75],ebp
00406C6F  C0E8 8B      shr     al,8B                        Shift constant out of range 1..31
00406C72  93           xchg    eax,ebx
00406C73  5F           pop     edi
00406C74  0130         add     [eax],esi
00406C76  8BC0         mov     eax,eax
00406C78  E8 83936101  call    01A20000
00406C7D  4B           dec     ebx
00406C7E  8BC0         mov     eax,eax
00406C80  50           push    eax
00406C81  6A 40        push    40
00406C83  E8 E0FFFFFF  call    00406C68
00406C88  C3           ret
```

Tiene muy mala pinta esto, si miramos la pila veremos que en realidad ya se han ejecutado unas cuantas instrucciones por lo que se trata del temido SCode, ahora estamos parados en lo que debería ser un salto a la IAT, miremos la pila:

Address	Value	Comment
0012FF78	014C113D	
0012FF7C	00000000	
0012FF80	3D83D8BF	
0012FF84	014C8C37	RETURN to 014C8C37 from 02F20000
0012FF88	012C03D0	
0012FF8C	3D83D8BF	
0012FF90	00000000	
0012FF94	012B0E50	
0012FF98	0051046F	RETURN to Useful_F.0051046F from Useful_F.0051046F
0012FF9C	00000000	
0012FFA0	0051086D	Useful_F.0051086D
0012FFA4	0012FF78	
0012FFA8	00560718	shell32.00560718
0012FFAC	0012FFF0	
0012FFB0	0012FFC4	
0012FFB4	7FFDF000	
0012FFB8	7FFE0304	
0012FFBC	0012FFB0	
0012FFC0	00000000	
0012FFC4	77E614C7	RETURN to kernel32.77E614C7 ← Programa inicia ejecución
0012FFC8	77F4164E	RETURN to ntdll.77F4164E from ntdll.77F4166D
0012FFCC	00560718	shell32.00560718
0012FFD0	7FFDF000	

El límite lo marca la posición [12FFC4] donde encontramos la dirección de retorno al Sistema Operativo como podemos comprobar fácilmente:

Address	Hex dump	Disassembly	Comment
77E614BC	6A FE	push -2	
77E614BE	FF15 9C13E477	call [<&ntdll.NtSetInfo	ntdll.ZwSetInformationThread
77E614C4	FF55 08	call [ebp+8]	
77E614C7	50	push eax	// Aqui retorna..
77E614C8	E8 C235FFFF	call ExitThread	
77E614CD	CC	int3	
77E614CE	8D88 FEEFFFFF	lea ecx,[eax-1002]	

Se ve después la llamada a ExitThread donde matará el proceso [Hilo - Thread] que es el propio programa, es la manera que tiene el S.O. de cargar todo programa, esto indica que los valores que hay entre [12FFC4] y [12FF78] son stolen bytes que forman parte del SCode, todas las instrucciones faltantes las ejecutó en la sección [014C0000] que por su puesto está fuera del EXE.

Para comprobar que esto es cierto entraremos con F7 en el "CALL 019C0000" y lo iremos traceando hasta retornar aquí:

Address	Hex dump	Disassembly	Comment
014C1137	5B	pop ebx	
014C1138	E9 2BA50000	jmp 014CB668	
014C113D	A3 18574D00	mov [4D5718],eax	Useful_F.00400000
014C1142	E9 B8930000	jmp 014CA4FF	
014C1147	84D2	test dl,dl	
014C1149	E8 B2EEA501	call 02F20000	
014C114E	F2:	prefix repne:	Superfluous prefix
014C114F	EB 01	jmp short 014C1152	

Antes de llegar a esta parte hemos pasado por dos secciones temporales [019C0000] y [019B0000] donde se ejecuta parte de la api [GetModuleHandleA] en este caso, y después retornamos a esta línea:

```
014C113D    A3 18574D00     mov     [4D5718],eax        ; Useful_F.00400000
```

Donde salva el valor hModule en [4D5718], una dirección que forma parte de una sección del EXE [justo después de ***data***] lo que indica que estamos en la sección donde ejecuta el ***SCode***, para ver donde realmente empieza a ejecutarse el programa [OEP] llegamos a la última excepción y colocamos un BPM en esta sección:

Address	Size	Owner	Section	Contains	Type	Access	Initial
014B0000	00001000				Priv	RWE	RWE
014C0000	00016000				Priv	RWE	RWE
014E0000	00001000						RWE
014F0000	00001000						RWE
01500000	00001000						RWE
01510000	00001000						RWE
01520000	00001000						RWE
01530000	00001000						RWE
01540000	00001000						RWE
01550000	00001000						RWE
01560000	00001000						RWE

Actualize
Dump in CPU
Dump
Search Ctrl+B
Set break-on-access F2
Set memory breakpoint on access
Set memory breakpoint on write

La pasamos con SHIFT+F9 y caemos en el verdadero OEP del programa:

```
Address   Hex dump      Disassembly                    Comment
014C029E  55            push    ebp
014C029F  26:EB 02      jmp     short 014C02A4         Superfluous prefix
014C02A2  CD 20         int     20
014C02A4  81ED AA51E4B1 sub     ebp,B1E451AA
014C02AA  F2:           prefix repne:                  Superfluous prefix
014C02AB  EB 01         jmp     short 014C02AE
014C02AD  698B ECE9D205 imul    ecx,[ebx+5D2E9EC],E9430000
014C02B7  9F            lahf
014C02B8  52            push    edx
014C02B9  0000          add     [eax],al
014C02BB  C1C9 61       ror     ecx,61                 Shift constant out
014C02BE  C1C9 DD       ror     ecx,0DD                Shift constant out
014C02C1  6A 50         push    50
014C02C3  EB 01         jmp     short 014C02C6
```

Ni que decir tiene que ese PUSH EBP debería estar dentro del ejecutable y no aquí, este es el citado SCode, hay muchas líneas de código que ejecuta en esta sección en lugar de hacerlo donde debería.

Os explicaré por encima lo que hace en esta zona, su estructura y cómo trabaja, después del 'PUSH EBP' inicial vemos un 'JMP + INT 20' totalmente innecesarios que tratan de obfuscar el código en la medida de lo posible, tonterías porque se ve perfectamente todo, pero bueno, después viene un 'SUB ESP, B1E451AA' si seguimos traceando al final nos toparemos con un call como este:

```
Address   Hex dump      Disassembly                    Comment
014CA6B4  68 378C4C01   push    14C8C37
014CA6B9  E8 4259A501   call    02F20000
014CA6BE  EB 01         jmp     short 014CA6C1
014CA6C0  F0:334424 08  lock xor eax,[esp+8]           LOCK prefix is not allowed
014CA6C5  B8 BA484400   mov     eax,4448BA
014CA6CA  8B06          mov     eax,[esi]
014CA6CC  68 64C84C01   push    14CC864
014CA6D1  E8 2A59A501   call    02F20000
014CA6D6  8D444B 29     lea     eax,[ebx+ecx*2+29]
014CA6DA  83E8 29       sub     eax,29
014CA6DD  64:EB 02      jmp     short 014CA6E2         Superfluous prefix
```

Más abajo se ve otro igual, y toda esta sección está llena y todos apuntan a una pequeña sección de tan solo 1000h bytes, entro en el call hasta encontrar esto:

```
Address   Hex dump      Disassembly                    Comment
02F20110  8D5475 1B     lea     edx,[ebp+esi*2+1B]
02F20114  8D91 98BC2A01 lea     edx,[ecx+12ABC98]
02F2011A  2BD1          sub     edx,ecx
02F2011C  FFD2          call    edx
02F2011E  2E:EB 01      jmp     short 02F20122         Superfluous prefix
02F20121  C7            ???                            Unknown command
02F20122  64:FF35 000000 push   dword ptr fs:[0]
```

Es un subcall, vuelvo a entrar [F7] y aterrizo al inicio de una función:

```
Address   Hex dump      Disassembly                    Comment
012ABC98  55            push    ebp                    // caigo aqui..
012ABC99  8BEC          mov     ebp,esp
012ABC9B  83C4 F8       add     esp,-8
012ABC9E  53            push    ebx
012ABC9F  56            push    esi
012ABCA0  57            push    edi
012ABCA1  8B5D 08       mov     ebx,[ebp+8]
012ABCA4  EB 01         jmp     short 012ABCA7
012ABCA6  9A 8B451883 E8 call   far 08E8:8318458B      Far call
012ABCAD  8B00          mov     eax,[eax]
```

Si continúo traceando con F8 llego a un bucle:

```
Address   Hex dump      Disassembly                    Comment
012ABCEF  8B45 F8       mov     eax,[ebp-8]            // inicio loop..
012ABCF2  0FB600        movzx   eax,byte ptr [eax]
012ABCF5  8B5483 44     mov     edx,[ebx+eax*4+44]
012ABCF9  8BC6          mov     eax,esi
012ABCFB  FFD2          call    edx
012ABCFD  3B45 FC       cmp     eax,[ebp-4]
012ABD00  75 1A         jnz     short 012ABD1C
012ABD02  8B45 10       mov     eax,[ebp+10]
012ABD05  50            push    eax
012ABD06  8B45 14       mov     eax,[ebp+14]
012ABD09  50            push    eax
012ABD0A  E8 CDF3FFFF   call    012AB0DC
012ABD0F  50            push    eax
012ABD10  8BCE          mov     ecx,esi
012ABD12  8B55 18       mov     edx,[ebp+18]
012ABD15  8BC3          mov     eax,ebx
012ABD17  E8 8CFDFFFF   call    012ABAA8
012ABD1C  4F            dec     edi
012ABD1D  0373 70       add     esi,[ebx+70]
012ABD20  85FF          test    edi,edi
012ABD22  77 CB         ja      short 012ABCEF         // fin loop..
```

Lo que hace en él es buscar un valor retornado en EAX a la salida del "call edx":

```
Address   Hex dump                                          ASCII
0012FEEC  67 9C 3C 5A 1A AA A1 01 1E 01 F2 02 54 A6 2D 01   g<Z
0012FEFC  B9 A6 45 01 06 02 00 00 0C FF 12 00 38 FF 12 00
0012FF0C  50 06 4D 00                         7F BF D8 83 3D
0012FF1C  38 FF 12 00                         00 00 29 01
0012FF2C  06 02 00 00 04 03 FE 7F BE A6 4C 01 37 8C 4C 01
0012FF3C  D0 03 2C 01 BF D8 83 3D 00 00 00 00 00 00 00 00
```

Valor buscado..

Seguirá loopeando mientras EAX!=5A3C9C67h, cuando lo tenga no se producirá el salto [JNZ] y se ejecutará el cuerpo del bucle, si en el segundo call de éste veo:

```
Address   Hex dump     Disassembly                  Comment
012ABC47    FF75 FC    push    dword ptr [ebp-4]
012ABC4A    FF75 10    push    dword ptr [ebp+10]
012ABC4D    FF75 0C    push    dword ptr [ebp+C]
012ABC50    8B45 F8    mov     eax,[ebp-8]
012ABC53  - FF60 24    jmp     [eax+24]             // Aqui termina..
012ABC56  v EB 01      jmp     short 012ABC59
012ABC58    9A 5F5E5B8B E5 call  far 5DE5:8B5B5E5F   Far call
012ABC5F    C2 0C00    ret     0C
```

Es el final de dicha función, si miro la ventanita auxiliar veo adonde saltará:

```
ds:[012DA678]=02F30000
```

Destino del salto

Otra sección nueva que no habíamos visto hasta ahora, si ponemos un HBP en el salto y traceamos algunas llamadas más descubrimos que este valor es fijo pues nunca varía, la nueva sección a la que salta [02F30000] termina con este salto:

```
Address   Hex dump     Disassembly                  Comment
02F300A1    8D52 EC    lea     edx,[edx-14]
02F300A4    5A         pop     edx
02F300A5    9D         popfd
02F300A6    5C         pop     esp
02F300A7  - FF6424 FC  jmp     [esp-4]              // Aqui termina..
02F300AB    36:EB 01   jmp     short 02F300AF       Superfluous prefix
02F300AE    F2:        prefix repne:                Superfluous prefix
```

Ese "JMP [ESP-4]" nos retorna a la sección del SCode justo aquí:

```
Address   Hex dump     Disassembly                  Comment
014CAAC9    01F2       add     edx,esi
014CAACB    8B06       mov     eax,[esi]
014CAACD  v E9 E3440000 jmp    014CEFB5
014CAAD2    C3         ret
014CAAD3    53         push    ebx                  <-- destino salto..
014CAAD4    BB 32DC4600 mov    ebx,46DC32
014CAAD9    F2:        prefix repne:                Superfluous prefix
014CAADA  v EB 01      jmp     short 014CAADD
```

Todos los cálculos que hizo fueron para determinar el punto a partir del cual proseguirá la ejecución dentro de esta misma zona, en lugar de haber un CALL directo pasa por esa rutina cada vez, si sigo traceando me encuentro otra llamada similar a la anterior:

```
Address   Hex dump     Disassembly                  Comment
014CAAEA    33C0       xor     eax,eax
014CAAEC    A3 10574D00 mov    [4D5710],eax
014CAAF1    6A 00      push    0
014CAAF3    68 3D114C01 push   14C113D
014CAAF8    E8 0355A501 call   02F20000             // segunda llamada..
014CAAFD    8D0411     lea     eax,[ecx+edx]
```

Por su puesto entro con F7 y sigo el mismo circuito de antes con la diferencia que ahora el "JMP [ESP-4]" me deja en el falso OEP:

```
Address   Hex dump     Disassembly                  Comment
00406C60    E8 9B935B01 call   019C0000
00406C65    3C 8B      cmp     al,8B
00406C67    C0E8 93    shr     al,93                Shift constant out
00406C6A    93         xchg    eax,ebx
00406C6B    5D         pop     ebp
00406C6C    0169 8B    add     [ecx-75],ebp
00406C6F    C0E8 8B    shr     al,8B                Shift constant out
```

Y este "CALL 019C0000" nos lleva a la primera api GetModuleHandleA, no hay que confundirlo con el SCode, aquí debería haber un "JMP DWORD PTR [PosIAT]" donde PosIAT sería la dirección de GMHandleA en la IAT, después veremos como recuperar

la iat y la tabla de saltos, pero eso será después, de momento nos centramos en el SCode, venimos de la segunda llamada a :02F20000 y al ejecutarse la api retornamos a dicha sección con EAX==400000 que es la imagebase del módulo actual [Useful File Utilities.exe], seguimos traceando hasta llegar a este call:

```
Address   Hex dump     Disassembly              Comment
014CD317   53          push    ebx
014CD318   56          push    esi
014CD319   57          push    edi
014CD31A   84D2        test    dl,dl
014CD31C   E8 DF2CA501 call    02F20000
014CD321   F2:         prefix repne:            Superfluous prefix
014CD322   EB 01       jmp     short 014CD325
```

Vemos algo distinto a los anteriores que iban precedidos de un PUSH mientras que en este caso tiene delante un "TEST DL,DL" si entramos en él vemos que es todo igual excepto al llegar aquí:

```
Address   Hex dump     Disassembly              Comment
012ABB0B   FFD2        call    edx              // retorna valor en AL
012ABB0D   2C 02       sub     al,2
012ABB0F   72 12       jb      short 012ABB23
012ABB11   74 43       je      short 012ABB56   // a zona nanomites..
012ABB13   FEC8        dec     al
012ABB15   0F84 8F000000 je    012ABBAA
012ABB1B   E9 EB000000 jmp     012ABC0B
```

Ese "CALL EDX" retorna un valor en AL que es verificado a la salida del call, en caso de retornar AL==02 no saltará en el JB y sí lo hará en el JE siguiente llegando a la zona "tipo nanomites" y no me he vuelto loco, jeje.. siguiendo el salto en que nos encontramos llegamos a esta zona:

```
Address   Hex dump     Disassembly              Comment
012ABB66   8BD8        mov     ebx,eax
012ABB68   8B4D 10     mov     ecx,[ebp+10]     // lee EFLAGS
012ABB6B   8BD3        mov     edx,ebx          // en EBX tipo salto
012ABB6D   8B45 F8     mov     eax,[ebp-8]
012ABB70   E8 BBFBFFFF call    012AB730         // lo revisa
012ABB75   84C0        test    al,al
012ABB77   74 1D       je      short 012ABB96   // se produce el salto?
012ABB79   EB 01       jmp     short 012ABB7C
```

Este tipo de call es típico de Armadillo, pone en ECX el valor de EFLAGS [Extended Flags] y en EDX el tipo de salto revisado, en esta ejecución tengo ECX==202 y EDX==04, si entro en el call me encuentro el típico salto indirecto:

```
Address   Hex dump     Disassembly              Comment
012AB739   33C0        xor     eax,eax
012AB73B   8AC2        mov     al,dl
012AB73D   83F8 0F     cmp     eax,0F
012AB740   0F87 F1010000 ja     012AB937
012AB746   FF2485 4DB72A0 jmp   [eax*4+12AB74D]  // decide la zona..
012AB74D   90          nop
```

Para EAX==04 nos deja en :012AB7DE donde hay un call al que se le pasan dos parámetros: los EFLAGS y una constante 0x06h como podemos ver:

```
Address   Hex dump     Disassembly              Comment
012AB7DE   6A 06       push    6                // Aqui si AL==04
012AB7E0   56          push    esi
012AB7E1   E8 4E83FFFF call    012A3B34         // se ejecuta el salto?
012AB7E6   8BD8        mov     ebx,eax
012AB7E8   E9 54010000 jmp     012AB941
```

Dentro del call encontramos esto:

```
Address   Hex dump     Disassembly              Comment
012A3B34   55          push    ebp
012A3B35   8BEC        mov     ebp,esp
012A3B37   52          push    edx
012A3B38   8B45 08     mov     eax,[ebp+8]      // lee EFLAGS
012A3B3B   8B55 0C     mov     edx,[ebp+C]      // lee valor 0x06h
012A3B3E   0FA3D0      bt      eax,edx          // flag/s activado/s?
012A3B41   19C0        sbb     eax,eax
012A3B43   83E0 01     and     eax,1            // obtiene resultado
012A3B46   5A          pop     edx
012A3B47   5D          pop     ebp
012A3B48   C2 0800     ret     8
```

En este momento verifica el valor de estos registros:

```
ECX==0x00000246h          // EFLAGS
EDX==0x00000006h          // binario: 00000110  [Banderas leídas]
```

Al final del call queda EAX==0x00000001h valor de retorno de la función:

Address	Hex dump	Disassembly	Comment
012ABB68	8B4D 10	mov ecx,[ebp+10]	// lee los EFLAGS
012ABB6B	8BD3	mov edx,ebx	// en EBX tipo salto
012ABB6D	8B45 F8	mov eax,[ebp-8]	
012ABB70	E8 BBFBFFFF	call 012AB730	
012ABB75	84C0	test al,al	
012ABB77	74 1D	je short 012ABB96	// No saltara..
012ABB79	EB 01	jmp short 012ABB7C	

No se produce el JE y llegamos al JMP que viene después y que nos pone en esta zona:

Address	Hex dump	Disassembly	Comment
012ABB7C	8B45 F8	mov eax,[ebp-8]	// El JMP viene aqui..
012ABB7F	8B40 1C	mov eax,[eax+1C]	
012ABB82	0345 F4	add eax,[ebp-C]	
012ABB85	8B55 F8	mov edx,[ebp-8]	
012ABB88	0342 6C	add eax,[edx+6C]	
012ABB8B	8945 F0	mov [ebp-10],eax	// Queda EAX==014C28B2h
012ABB8E	E9 80000000	jmp 012ABC13	
012ABB93	EB 01	jmp short 012ABB96	

Tras ejecutar las líneas hasta el JMP ha calculado un valor en EAX que será el lugar de destino, en este caso quedó EAX==0x014C28B2h donde hay esto:

Address	Hex dump	Disassembly	Comment
014C28B2	84D2	test dl,dl	<-- destino..
014C28B4	E8 47D7A501	call 02F20000	
014C28B9	2BCB	sub ecx,ebx	
014C28BB	2E:EB 01	jmp short 014C28BF	Superfluous prefix
014C28BE	C7	???	Unknown command

O sea que proseguirá la ejecución a partir de este nuevo punto, en caso de ejecutarse el JE de :012ABB77 saltaría a esta otra zona:

Address	Hex dump	Disassembly	Comment
012ABB96	8B45 F8	mov eax,[ebp-8]	// JE se ejecuta..
012ABB99	0378 1C	add edi,[eax+1C]	
012ABB9C	8B45 F8	mov eax,[ebp-8]	
012ABB9F	0378 6C	add edi,[eax+6C]	
012ABBA2	897D F0	mov [ebp-10],edi	// destino en EDI
012ABBA5	EB 6C	jmp short 012ABC13	
012ABBA7	EB 01	jmp short 012ABBAA	

Donde hace algo similar y coloca el nuevo valor de destino también en [ebp-10], continuamos traceando y vemos que efectivamente llegamos al lugar esperado 014C28B2 donde hay un nuevo "TEST DL,DL" y un "call 02F20000" donde se repite la jugada de leer los EFLAGS, de nuevo el resultado es EAX==0x01 a la salida del call en :012ABB70 llegamos a la zona de antes donde deja EAX==0x014CF62Ah donde sólo hay un RET el cual nos deja aquí:

Address	Hex dump	Disassembly	Comment
0040DEA8	8BF0	mov esi,eax	
0040DEAA	33D2	xor edx,edx	
0040DEAC	8BC6	mov eax,esi	
0040DEAE	E8 555BFFFF	call 00403A08	// estabamos dentro
0040DEB3	C746 0C FFFF00	mov dword ptr [esi+C],0FFFF	// Aqui salimos
0040DEBA	6A 00	push 0	
0040DEBC	6A FF	push -1	
0040DEBE	6A FF	push -1	
0040DEC0	6A 00	push 0	
0040DEC2	E8 CD8FFFFF	call 00406E94	
0040DEC7	8946 10	mov [esi+10],eax	
0040DECA	6A 00	push 0	

Aterrizamos nuevamente en el código del EXE, qué pasada! esto no para de dar saltos de arriba para abajo, jeje.. si me pongo sobre el call de donde se supone que venimos y aprieto ENTER veo esto:

Address	Hex dump	Disassembly	Comment
00403A05	C3	ret	
00403A06	8BC0	mov eax,eax	
00403A08 -	E9 3AD70B01	jmp 014C1147	// Pedazo de salto!!
00403A0D	0FBAE5 B3	bt ebp,0B3	
00403A11	2886 F7A499D0	sub [esi+D099A4F7],al	
00403A17	57	push edi	
00403A18	58	pop eax	

Hay un salto directo a la zona del packer donde hay otro "TEST DL,DL" y una llamada "CALL 02F20000" similar a las dos que acabamos de ver, pero.. esa zona no se ejecutó ¿? estoy en un laberinto de llamadas de este estilo que me rebotan de un lado para otro sin parar, a veces aterrizo en el ejecutable y otras veces

no, bouufff!!! esto está descarriado, es un sinfín de llamadas concatenadas las unas con las otras con entradas intermitentes al EXE protegido.

Pero no todo va a ser negativo pues de lo visto deducimos que los "CALL 02F20000" reemplazan en el código original del ejecutable a:

1.- Otras llamadas [CALL xxxxxxxx]
2.- Saltos condicionales [JE/JNE/JG/JL...]

Claro que podríamos ir descifrando uno por uno estos call y modificarlos pero sería una tarea titánica, un mejor método sería injertar el código "robado" en el ejecutable y allocarlo [como dice Ricardo] después en memoria con VirtualAlloc. Pero se nos plantea un problema ¿tenemos espacio para hacer esto? es cuestión de mirarlo y de utilizar ToPo v1.2 si hiciera falta.

De momento nos preocuparemos de listar las secciones necesarias para el futuro archivo desempakado, son éstas:

1.- [dirección: 01290000 - tamaño: 2C000h bytes]
2.- [dirección: 012C0000 - tamaño: 4000h bytes]
3.- [dirección: 012D8000 - tamaño: 4000h bytes]
4.- [dirección: 014C0000 - tamaño: 16000h bytes]
5.- [dirección: 0012C000 - tamaño: 4000h bytes] // Zona de pila [stack]

Son cuatro secciones más el espacio de pila que posiblemente no nos haga falta injertar pero que igualmente lo haremos por si las moscas. Ahora que sabemos donde están las instrucciones faltantes debemos buscar la manera de hacer que se ejecuten en el EXE desempakado, no sin antes arreglar la tabla [iat] que está bastante tocada, al igual que la tabla de saltos.

Restaurando tablas..

Ya que son dos [IAT y JMP Table], si hemos desempakado alguna vez un ASProtect sabremos que modifica la iat para que no funcione un posible dumpeado que obtengamos a partir del ejecutable protegido, las últimas versiones hacen lo mismo pero además de ubicar la iat fuera del EXE [una especie de IAT Scrambled] modifica algunos saltos tipo 0xFF25h por CALLs que apuntan a secciones virtuales creadas en tiempo de ejecución por medio de VirtualAlloc, el método es conocido por nosotros, os remito a las teorías del grupo si tenéis alguna duda, o incluso me podéis escribir a mi e-mail, en los tutes de ASProtect es donde encontraréis cómo reparar una iat y una tabla de saltos, a partir de las versiones 1.2x-1.3x, 2.0x y 2.1 SKE es posible encontrarse esta protección. En mi tute "***Estudio de ASProtect 1.2x - 1.3x [Registered]***" podéis encontrar información al respecto, incluso podemos utilizar el script que se utilizó allí ya que es muy similar pues se trata también de ASProtect.

Antes de entrar en materia necesitamos algunos datos que vamos a buscar ahora mismo, reiniciamos Olly con CTRL+F2 y llegamos al falso OEP, os facilito un pequeño script que nos facilitará enormemente este trabajo:

```
Inicio:
dbh                 // oculto el debugger
eob Vamos           // si BP/Excep salta a "vamos"
run                 // dejo correr el programa

Vamos:
cmp eip, 012AFB34   // Es la última excepción?
je Ultima           // Si es así va a "Ultima"
esti                // Equivale a SHIFT+F7
jmp Inicio          // Al inicio del script
```

```
Ultima:
bprm 00406C60, 04     // BPM de lectura en el OEP
esti                  // Equivale a SHIFT+F7
eob Termina           // si BP/Excep salta a "Termina"
run                   // Dejo correr el programa

Termina:
cmp eip, 012AFDC3     // Cambió la última excepción
je Otra               // Si fue así salta a "Otra"
bpmc                  // Quito el BPM
ret                   // Termina el script

Otra:
esti                  // Equivale a SHIFT+F7
eob Termina           // si BP/Excep salta a "Termina"
run
```

No necesita comentarios, es muy simple, los tres valores que necesita son: el OEP [406C60] que en realidad es el falso OEP, la dirección de la última excepción [012AFB34], y como sabemos que la última excepción a veces pasa a ser la penúltima, se la pasamos también por si se diera el caso [012AFDC3], y ya está, desde el EP con las excepciones activadas lo lanzamos y estaremos en el OEP en un par de segundos, y ahora sí que es el momento de estudiar el tema IAT que encontramos buscando el Opcode 0xFF25, pulsamos CTRL+B ingresamos dicho valor y caemos aquí:

Address	Hex dump	Disassembly		Comment
00401260	- FF25 94782F01	jmp	[12F7894]	kernel32.CloseHandle
00401266	8BC0	mov	eax,eax	
00401268	- FF25 A0782F01	jmp	[12F78A0]	kernel32.CreateFileA
0040126E	8BC0	mov	eax,eax	
00401270	E8 8BED0F01	call	01500000	
00401275	138B C0E883ED	adc	ecx,[ebx+ED83E8C0]	
0040127B	1101	adc	[ecx],eax	
0040127D	C9	leave		
0040127E	8BC0	mov	eax,eax	
00401280	E8 7BED1301	call	01540000	
00401285	088B C0E873ED	or	[ebx+ED73E8C0],cl	
0040128B	15 01398BC0	adc	eax,C08B3901	
00401290	- FF25 DC782F01	jmp	[12F78DC]	kernel32.RaiseException
00401296	8BC0	mov	eax,eax	
00401298	- FF25 E8782F01	jmp	[12F78E8]	kernel32.ReadFile
0040129E	8BC0	mov	eax,eax	
004012A0	- FF25 F4782F01	jmp	[12F78F4]	
004012A6	8BC0	mov	eax,eax	
004012A8	E8 53ED1801	call	01590000	

Hacemos un FOLLOW IN DUMP - SELECTION para ver los bytes del salto:

Address	Hex dump	ASCII
00401260	FF 25 94 78 2F 01 8B C0 FF 25 A0 78 2F 01 8B C0	ÿ%"x/.‹Àÿ% x/.‹À
00401270	E8 8B ED 0F 01 13 8B C0 E8 83 ED 11 01 C9 8B C0	è‹í¤.".‹Àèƒí◄.É‹À
00401280	E8 7B ED 13 01 00 HBPM En estos bytes 8B C0	è{í".□‹Àèsí↨.9‹À
00401290	FF 25 DC 78 2F 01 8B C0 FF 25 E8 78 2F 01 8B C0	ÿ%Ux/.‹Àÿ%èx/.‹À
004012A0	FF 25 F4 78 2F 01 8B C0 E8 53 ED 18 01 C9 8B C0	ÿ%ôx/.‹ÀèSí↑.É‹À
004012B0	FF 25 0C 79 2F 01 8B C0 FF 25 18 79 2F 01 8B C0	ÿ%.y/.‹Àÿ%↑y/.‹À
004012C0	E8 3B ED 1B 01 9C 8B C0 FF 25 30 79 2F 01 8B C0	è;í←.œ‹Àÿ%0y/.‹À

Ponemos un BREAKPOINT - HARDWARE, ON ACCESS - WORD ya que no nos deja ponerlo tipo DWORD, reiniciamos Olly y damos RUN [F9] pero.. ¡no para!, quitamos el HBP y ponemos un BREAKPOINT - MEMORY, ON WRITE, pulsamos F9 y tras un par de paradas sin importancia llegamos a la zona donde escribe los bytes finales del salto:

Address	Hex dump	Disassembly		Comment
012A5E49	8928	mov	[eax],ebp	
012A5E4B	8B4424 0C	mov	eax,[esp+C]	
012A5E4F	8906	mov	[esi],eax	// en EAX los bytes!
012A5E51	0FB74424 04	movzx	eax,word ptr [esp+4]	
012A5E56	0143 08	add	[ebx+8],eax	
012A5E59	^ E9 B1FCFFFF	jmp	012A5B0F	// al punto gatillo..

Este punto es muy importante porque tenemos en EAX los bytes finales del salto, además tenemos en EBP la api correcta para dicho salto y en ESI la dirección de la JMP Table donde irán los bytes que tiene EAX, veamos una instantánea mejor:

```
Registers (FPU)
EAX 012F7894
ECX 77F432FA ntdll.77F432FA
EDX 77FB306C ntdll.77FB306C
EBX 012C03E0
ESP 0012FEF8
EBP 77E5A6F0 kernel32.CloseHandle
ESI 00401262 Useful_F.00401262
EDI 012F7860
```

Como vimos antes el salto está en :401260 y los dos primeros bytes son el Opcode de salto 0xFF25h que ocupa dos bytes, los cuatro restantes están reservados para los bytes que marcarán el destino del mismo y será :401262 en este salto, valor que encontramos en ESI, esto es interesante de cara al script que modificará el salto para que apunte a la iat allí donde quede guardada la api asociada [CloseHandle], dirección que encontraremos en EBP. Lo tenemos todo para reparar los saltos que se escriban aquí ya que tiene otras zonas, si ponemos un BPM en toda la sección de código en escritura [ON WRITE] irá parando en estos sitios:

```
Address   Hex dump    Disassembly                      Comment
012A3AE9  E8 56FFFFFF call    012A3A44
012A3AEE  C603 E8     mov     byte ptr [ebx],0E8       // Opcode de CALL
012A3AF1  43          inc     ebx
012A3AF2  8903        mov     [ebx],eax                // bytes del mismo
012A3AF4  5B          pop     ebx
012A3AF5  C3          ret
```

Aquí para dos o tres entradas después y es la zona donde inserta los CALLs cuyo Opcode es el 0x0E8h que vemos en la imagen, si echamos un vistazo a la ventana de registros vemos los valores que necesitamos:

```
Registers (FPU)
EAX 010FED8B
ECX 0012FF01
EDX 010FED8B
EBX 00401270 Useful_F.00401270
ESP 0012FEF0
EBP 77E5E3FB kernel32.GetFileType
ESI 00401272 Useful_F.00401272
EDI 012F7860
```

Como antes tenemos la api en EBP y en ESI la posición donde salvar la dirección de la iat donde quede la api, aquí hay una cosita más pues habrá además que insertar el Opcode de salto puesto que no está escrito:

```
Address   Hex dump                                         ASCII
00401270  00 00 1E 61 AD 13 8B C0 00 00 9B E1 6E C9 8B C0  ..a-¶‹À..›ánÉ‹À
00401280  00 00 0C D8 6E 08 8B C0 00 00 41 EC A8 39 8B C0  ...Ø^◘‹À..Aì¨9‹À
00401290  FF 25 4A A5 2E AC 8B C0 FF 25 77 E6 7B EC 8B C0  ÿ%J¥.¬‹Ày%wæ{ì‹À
004012A0  FF 25 58 60 01 bytes del salto C5 7C C9 8B C0  ÿ%X`☺u‹À..½Å|É‹À
004012B0  FF 25 B6 D4 55 63 8B C0 FF 25 93 83 13 90 8B C0  ÿ%¶ÔUc‹Ày%"f¶◘‹À
004012C0  00 00 Opcode 0xFF25h FF 25 79 7D E3 9F 8B C0  ..ãáÛœ‹Ày%y}ãŸ‹À
004012D0  00 00 62 5B F4 27 8B C0 FF 25 EF D4 FC B9 8B C0  ..b[ô'‹Ày%ïÔü'‹À
004012E0  FF 25 B0 E8 3B 27 8B C0 00 00 75 CF AE 75 8B C0  ÿ%'è;'‹À..uI®u‹À
```

Ahí se ve que en lugar del famoso 0xFF25h hay un 0x0000h, esto hay que tenerlo en cuenta también para el script que repare la tabla.

Pero todo no termina aquí porque si seguimos pulsando F9 parará en una tercera zona donde modifica otros saltos:

```
Address   Hex dump    Disassembly                      Comment
012A5D4D  8902        mov     [edx],eax
012A5D4F  8B4424 0C   mov     eax,[esp+C]
012A5D53  8906        mov     [esi],eax                // en EAX los bytes!
012A5D55  0FB74424 04 movzx   eax,word ptr [esp+4]
012A5D5A  0143 08     add     [ebx+8],eax
012A5D5D  E9 ADFDFFFF jmp     012A5B0F                 // al punto gatillo..
```

Estamos en el mismo caso que el de :012A5E4F en lo que a registros se refiere, mejor para nosotros ya que el script trabajará igual en ambos casos, después para en este otro punto que puede inducir a error:

```
Address   Hex dump    Disassembly                      Comment
012A5E9A  894424 0C   mov     [esp+C],eax
012A5E9E  8B4424 0C   mov     eax,[esp+C]
012A5EA2  8906        mov     [esi],eax                // caso GetProcAddress
012A5EA4  8A4C24 06   mov     cl,[esp+6]
012A5EA8  8B5424 14   mov     edx,[esp+14]
012A5EAC  8B4424 0C   mov     eax,[esp+C]
012A5EB0  E8 FFF0FFFF call    012A4FB4
012A5EB5  E9 55FCFFFF jmp     012A5B0F                 // al punto gatillo..
```

Este es más delicado pues a simple vista parece igual a los anteriores donde tenemos la api en EBP y en ESI la posición donde salvar la dirección de la iat donde quedó la api almacenada, si miramos los registros vemos esto:

```
Registers (FPU)
EAX 012F79D8
ECX 012F79D8
EDX 00000000
EBX 012C03E0
ESP 0012FEF8
EBP 77E5AD86 kernel32.GetModuleHandleA
ESI 0040133A Useful_F.0040133A
EDI 012F7860
```

Todo apunta a que se trata de GetModuleHandleA pero en realidad no es así, si seguimos pulsando F9 veremos que aquí solamente para una vez más, si vamos a la dirección que marca EAX vemos que hay una dirección:

```
Address   Hex dump                                          ASCII
012F79D8  B8 46 2A 01 00 00 00 00 0E 00 00 00 00 00 76 01  ¸F* ┌....♪.....v┌
012F79E8  00 00 00 00 0E 00 00 00 00 00 78 01 00 00 00 00  ....♪.....x┌....
012F79F8  0E 00 00 00 00 00 7A 01 00 00 00 00 0E 00 00 00  ♪.....z┌....♪...
012F7A08  00 00 7B 01 00 00 00 00 0E 00 00 00 00 00 7C 01  ..{┌....♪.....|┌
012F7A18  00 00 00 00 0E 00 00 00 00 00 7D 01 00 00 00 00  ....♪.....}┌....
012F7A28  0E 00 00 00 00 00 7E 01 00 00 00 00 0E 00 00 00  ♪.....~┌....♪...
```

Si seleccionamos los 4 bytes marcados en amarillo, picamos con el botón derecho y elegimos FOLLOW DWORD IN DUMP caeremos en dicha dirección [012A46B8], si ponemos ahora la vista en modo DISASSEMBLE encontraremos lo que buscamos:

```
Address   Hex dump     Disassembly                  Comment
012A46B8  55           push   ebp
012A46B9  8BEC         mov    ebp,esp
012A46BB  8B55 0C      mov    edx,[ebp+C]
012A46BE  8B45 08      mov    eax,[ebp+8]
012A46C1  3B05 F8332B01 cmp   eax,[12B33F8]
012A46C7  75 09        jnz    short 012A46D2
012A46C9  8B0495 F8332B01 mov eax,[edx*4+12B33F8]
012A46D0  EB 07        jmp    short 012A46D9
012A46D2  52           push   edx
012A46D3  50           push   eax
012A46D4  E8 2710FFFF  call   01295700            jmp to kernel32.GetProcAddress
012A46D9  5D           pop    ebp
012A46DA  C2 0800      ret    8
```

Esta api siempre dando problemas, grrr.. lógicamente se trata de GetProcAddress y la repararemos después a mano, por ahora seguimos dando a F9 y encontramos una zona diferente a todas las anteriores:

```
Address   Hex dump   Disassembly                Comment
014E03C1  5F         pop    edi
014E03C2  037B 04    add    edi,[ebx+4]
014E03C5  890F       mov    [edi],ecx           // en ECX los bytes!
014E03C7  51         push   ecx
014E03C8  B9 56A04200 mov   ecx,42A056
014E03CD  034C24 38  add    ecx,[esp+38]
014E03D1  64:EB 02   jmp    short 014E03D6      Superfluous prefix
```

En este caso varía un poco el esquema, los bytes los tenemos en ECX y no en EAX, esta es la primera diferencia, si miramos los registros vemos:

```
Registers (FPU)
EAX 770F0000 offset oleaut32.#392
ECX 012F7A8C
EDX 770F0000 offset oleaut32.#392
EBX 0012FF28
ESP 0012FEB0
EBP 0012FED8
ESI 770FF42E oleaut32.SysReAllocStringLen
EDI 004013B2 Useful_F.004013B2
```

La api está en ESI y no en EBP, además la dirección donde salvaremos la posición de la iat donde quedó la api está en EDI y no en ESI, como cambia todo el script tratará esta zona de manera distinta.

Después ya no para en ninguna zona distinta de las cinco que hemos visto para tratar más saltos a la iat, pero lo que sí hace es parar en dos nuevas zonas donde inserta más basura en el EXE, esta es la primera:

```
Address   Hex dump    Disassembly                  Comment
012AC456  C600 68     mov     byte ptr [eax],68    // inserta un PUSH
012AC459  8D50 01     lea     edx,[eax+1]
012AC45C  8932        mov     [edx],esi            // lo reajusta
012AC45E  83C0 05     add     eax,5
012AC461  C600 C3     mov     byte ptr [eax],0C3   // inserta un RET
012AC464  8A45 FF     mov     al,[ebp-1]
012AC467  5F          pop     edi
012AC468  5E          pop     esi
012AC469  5B          pop     ebx
012AC46A  59          pop     ecx
012AC46B  5D          pop     ebp
012AC46C  C2 0400     ret     4
```

Inserta una combinación tipo PUSH+RET en el ejecutable desviando la ejecución hacia una sección virtual como podemos ver:

```
Address   Hex dump    Disassembly                  Comment
00408D1C  68 B88F2D01 push    12D8FB8              // push insertado
00408D21  C3          ret                          // ret
00408D22  0C CE       or      al,0CE
00408D24  8388 EE410337 [ or  dword ptr [eax+3703
00408D2B  62E7        bound   esp,edi              Illegal use of register
00408D2D  4F          dec     edi
00408D2E  B8 F4F459DD mov     eax,DD59F4F4
```

En esta imagen se aprecia el "PUSH+RET" que nos enviará a la dirección que marca dicho PUSH [012D8FB8] si nos fijamos estamos en :00408D1C que pertenece al ejecutable, este será otro problema con el que debemos lidiar, pero aún hay más:

```
Address   Hex dump    Disassembly                  Comment
012A3ACB  C603 E9     mov     byte ptr [ebx],0E9   // inserta un JMP
012A3ACE  8D53 01     lea     edx,[ebx+1]
012A3AD1  8902        mov     [edx],eax            // y lo reajusta..
012A3AD3  8B45 08     mov     eax,[ebp+8]
012A3AD6  8910        mov     [eax],edx
012A3AD8  B8 05000000 mov     eax,5
012A3ADD  5B          pop     ebx
012A3ADE  5D          pop     ebp
012A3ADF  C2 0400     ret     4
```

Aquí inserta algunos saltos directos a una zona distinta similar a la anterior, si hacemos FOLLOW IN DUMP - MEMORY ADDRESS vemos el salto que pone:

```
Address   Hex dump    Disassembly                  Comment
00403A08  - E9 3AD70B01 jmp   014C1147             // Pedazo de salto!!
00403A0D  0FBAE5 B3   bt      ebp,0B3
00403A11  2886 F7A499D0 sub   [esi+D099A4F7],al
00403A17  57          push    edi
00403A18  58          pop     eax
00403A19  B2 52       mov     dl,52
00403A1B  F4          hlt                          Privileged command
```

Lo dicho.. más basura ;(

Y después de ver los destrozos que hace el packer en el ejecutable podemos intentar detenerle, jeje.. el trabajo será duro pero no imposible por su puesto. Lo primero que haremos será recuperar la iat y la tabla de saltos, para esto tenemos el script ya programado del tute citado anteriormente al que sólo hay que cambiarle algunas direcciones y retocarlo un poco. Es éste:

```
//*********************************************************
//* Script para ASProtect 1.2x-1.3x [Registered]
//* Objetivos: Recuperar la iat y la tabla de saltos
//* Author: SyXe'05 .::[CracksLatinoS]::.
//* Email: syxe05@gmail.com
//* S.O: WinXP SP1, OllyDbg v1.10, OllyScript v0.92
//*********************************************************

var Flag                  // Actuará como una bandera
var Api                   // Contendrá la api tratada
var Base                  // Tendrá la base de la DLL actual
var Direc                 // Dirección JMP/CALL a reparar
var PosIat                // Donde quedó la api salvada

bphws 012A3AEE, "x"       // Colocamos el primer HBP
bphws 012A5E4F, "x"       // Colocamos el tercer HBP
bphws 012A5D53, "x"       // Colocamos el cuarto HBP
bphws 014E03C5, "x"       // Colocamos el segundo HBP
```

```
mov PosIat, 4DA750          // Aquí ubicaremos la tabla
mov Flag, 0                 // Inicializamos la bandera

Inicio:
eob Vamos                   // Si Excep/BP salta a "Vamos"
run                         // dejamos que corra..

Vamos:
cmp eip, 012A3AEE           // Estamos en la zona de CALLs?
je Calls                    // Si es así salta a Calls
cmp eip, 014E03C5           // Estamos en la zona 014E03C5?
je IATScrambled             // Si es así salta a IATScrambled
cmp eip, 012A5E4F           // Estamos en la zona de saltos?
je JMPs                     // Si es así salta a JMPs
cmp eip, 012A5D53           // Estamos en la zona de saltos?
je JMPs                     // Si es así salta a JMPs
jmp Termina                 // Sino termina el script

// Procedimiento Calls
Calls:
mov Api, ebp                // Guardamos la Api
log Api                     // La logueamos
cmp Flag, 0                 // Es la primera vez?
jne OtraVez                 // Salvamos directamente en la iat
GMI Api, MODULEBASE         // Obtenemos su imagebase
mov Base, $RESULT           // Y la guardamos en "Base"

Iguales:
mov [PosIat], Api           // Guardamos la Api en la iat
mov  [ebx], #FF25#          // Reparamos el call
add ebx, 2                  // Reajustamos el puntero
mov  [ebx], PosIat          // Lo dejamos apuntando a la iat
log PosIat                  // Logueamos su valor
add PosIat, 4               // Apuntamos al siguiente DWORD
mov Flag, 1                 // Activamos la bandera
mov eip, 012A3AF4           // Salteamos el EIP
jmp Inicio                  // Y salta al inicio del script

OtraVez:
GMI Api, MODULEBASE         // Obtenemos su imagebase
cmp Base, $RESULT           // Y la comparamos con "Base"
je Iguales                  // Salta si apis del mismo tipo
mov Base, $RESULT           // Sino actualizamos la variable
add PosIat, 4               // Dejamos una separación de ceros
jmp Iguales                 // Y Vamos a salvarla igualmente

// Procedimiento IATScrambled
IATScrambled:
mov Api, esi                // Guardamos la Api en su variable
log Api                     // Y la logueamos
cmp Flag, 0                 // Miramos si es la primera pasada
jne NoEs                    // Saltará si no es la primera
GMI api, MODULEBASE         // Obtenemos su imagebase
mov Base, $RESULT           // Y la guardamos en "Base"

Guardo:
mov [PosIat], Api           // Guardamos la Api en la iat
mov Direc, edi              // Dirección de la JMP Table
sub Direc, 2                // Ajustamos para insertar Opcode
mov [Direc], #FF25#         // y lo insertamos en su lugar
```

```
mov [edi], PosIat          // Reparamos el salto a la iat
log PosIat                 // Logueamos su valor
add PosIat, 4              // Apuntamos al siguiente DWORD
mov Flag, 1                // Activamos la bandera
mov eip, 014E03C7          // Modificamos el EIP
jmp Inicio                 // Saltamos al inicio del script

NoEs:
GMI Api, MODULEBASE        // Obtenemos su imagebase
cmp Base, $RESULT          // Y la comparamos con "Base"
je Guardo                  // Salta si apis del mismo tipo
mov Base, $RESULT          // Sino actualizamos la variable
add PosIat, 4              // Dejamos una separación de ceros
jmp Guardo                 // Y ahora sí Vamos a guardarla

// Procedimiento JMPs
JMPs:
mov Api, ebp               // Guardamos la Api en su variable
log Api                    // Y la logueamos
cmp Flag, 0                // Miramos si es la primera pasada
jne Sgute                  // Saltará si no es la primera
GMI Api, MODULEBASE        // Obtenemos su imagebase
mov Base, $RESULT          // Y la guardamos en "Base"

Salvo:
mov [PosIat], Api          // Guardamos la Api en la iat
mov [esi], PosIat          // Reparamos el salto a la iat
log PosIat                 // Logueamos el destino del salto
add PosIat, 4              // Apuntamos al siguiente DWORD
mov Flag, 1                // Activamos la bandera
add eip, 2                 // Modificamos el EIP
jmp Inicio

Sgute:
GMI Api, MODULEBASE        // Obtenemos su imagebase
cmp Base, $RESULT          // Y la comparamos con "Base"
je Salvo                   // Salta si apis del mismo tipo
mov Base, $RESULT          // Sino actualizamos la variable
add PosIat, 4              // Dejamos una separación de ceros
jmp Salvo                  // Y Vamos a salvarla

// Etiqueta Paso
Paso:
esti                       // Shift+F7
jmp Inicio                 // Saltamos al Inicio del script

// Terminamos el trabajo
Termina:
cmp eip, 012A4C86         // Excepión de GetProcAddress?
je Paso                   // Si es así la pasamos
cmp eip, 012A4CC9         // Excepión de GetProcAddress?
je Paso                   // Si es así la pasamos
msg "trabajo terminado ;)"// Mostramos mensaje por pantalla
ret                       // Termina el script :)
```

Básicamente lo que hace es poner un HBP en cada una de las zonas donde trata los saltos a la iat, a excepción de la que maneja GetProcAddress que arreglaremos después a mano en el OEP, como se producen excepciones durante el desarrollo agregamos dos chequeos al final para ver si realmente terminó el trabajo o se

detuvo antes, estas excepciones son dos y se generan cada vez que trata uno de los dos saltos a GetProcAddress que tiene que ser especial para todo. El resto ya está comentado al margen, adjunto el script al tute.

Pero antes de lanzarlo a lo bruto hay que pulir unos asuntos, pues el objetivo es meter en el ejecutable el máximo de código del *SCode* y con el script nos plantamos prácticamente en el OEP que está tres o cuatro excepciones después de que termine, lo que yo hice fue ir probando a desviar secciones de las necesarias al ejecutable con el inconveniente que no podemos utilizar una zona utilizada como buffer y que aparentemente está llena de ceros, os ahorro los hostiazos que me di y pongo una forma que me aceptó. Coseguí salvar dos secciones que son las dos más pequeñas, pues las de 16000h bytes y 2C000h bytes no las aceptaba de ninguna manera, conseguí salvarlas así:

- orígen: 02F20000 / tamaño: 1000h bytes --> destino: 4DC000
- orígen: 02F30000 / tamaño: 1000h bytes --> destino: 4DE000

La forma en que se desvían es la más fácil, ahora sí lanzamos el script, el punto gatillo es éste:

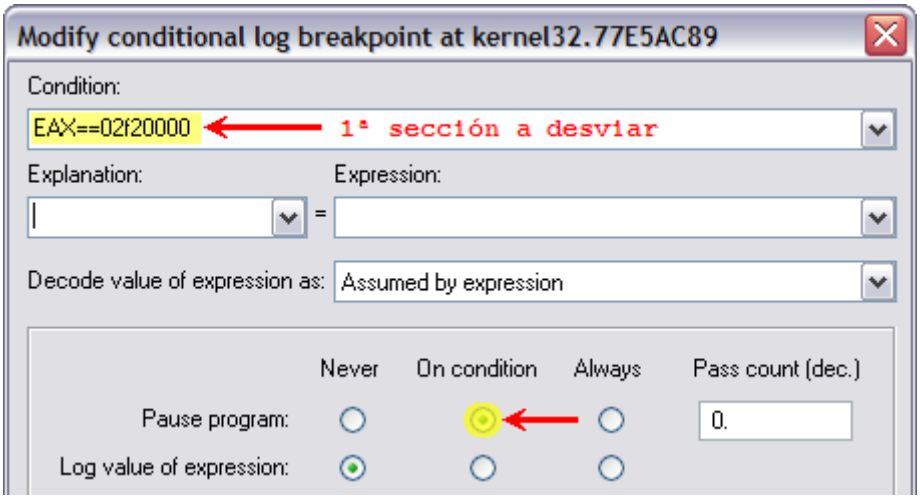
```
Address   Hex dump     Disassembly                      Comment
012A5B0F  8B43 08      mov     eax,[ebx+8]              // punto gatillo..
012A5B12  8B30         mov     esi,[eax]
012A5B14  8343 08 04   add     dword ptr [ebx+8],4
012A5B18  8B43 08      mov     eax,[ebx+8]
012A5B1B  8A00         mov     al,[eax]
012A5B1D  884424 07    mov     [esp+7],al
012A5B21  FF43 08      inc     dword ptr [ebx+8]
012A5B24  85F6         test    esi,esi
012A5B26  75 1A        jnz     short 012A5B42
012A5B28  EB 01        jmp     short 012A5B2B
```

Si nos fijamos algunas zonas donde trataba los saltos a la iat terminaban con un "JMP 012A5B0F" y este es un buen punto para lanzar el script, lo que haremos será partir desde el EntryPoint [EP] pulsar F9, superar un par de excepciones con SHIFT+F9, ir hasta el punto gatillo con CTRL+G y poner un BP en él, superar la excepción en la que estamos y llegamos hasta allí, podemos ir superando las excepciones que van saltando o bien deshabilitarlas momentáneamente, cada cual lo puede hacer como quiera, pero al llegar al gatillo hay que habilitarlas para el script, una vez en :012A5B0F con las opciones habilitadas entonces sí lo lanzamos PLUGINS - OLLYSCRIPT - RUN SCRIPT.. pero falla:

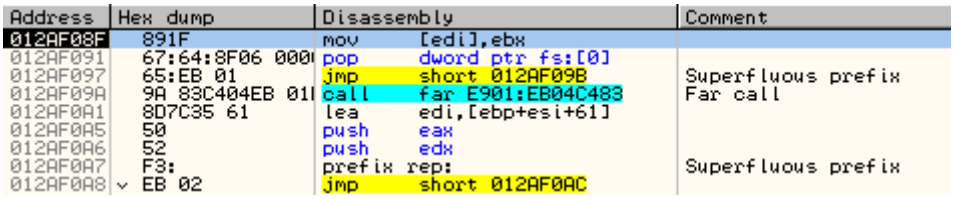
Parece el típico error de ASProtect, sabemos que es muy quejoso y que tiene innumerables chequeos de código, incluso a veces no se queja pero llega un punto en que deja de funcionar correctamente:

```
Address   Hex dump     Disassembly                      Comment
00401310  - FF25 C0A74D00 jmp     [4DA7C0]              kernel32.GetCurrentDirectoryA
00401316    8BC0         mov     eax,eax
00401318  - FF25 C8A74D00 jmp     [4DA7C8]              ntdll.RtlGetLastWin32Error
0040131E    8BC0         mov     eax,eax
00401320  - FF25 D0A74D00 jmp     [4DA7D0]              kernel32.GetLocaleInfoA
00401326    8BC0         mov     eax,eax
00401328  - FF25 D4A74D00 jmp     [4DA7D4]              kernel32.GetModuleFileNameA
0040132E    8BC0         mov     eax,eax
00401330  - FF25 D8A74D00 jmp     [4DA7D8]              kernel32.GetModuleHandleA
00401336    8BC0         mov     eax,eax
00401338  - FF25 D8792F01 jmp     [12F79D8]             // ya no funka ;(
0040133E    8BC0         mov     eax,eax
00401340    E8 BBEC3501  call    01760000
00401345  ^ 7D 8B        jge     short 004012D2
00401347    C0E8 B3      shr     al,0B3                Shift constant out of range 1..
0040134A    EC           in      al,dx                 I/O command
0040134B    37           aaa
0040134C    01C1         add     ecx,eax
0040134E    8BC0         mov     eax,eax
00401350    E8 ABEC3901  call    017A0000
00401355    698B C0FF2508 imul   ecx,[ebx+825FFC
```

Parece ir todo bien pero al llegar al salto de :00401338 falla, si nos fijamos en el Log vemos que se trata de GetModuleHandleA según pone, pero sabemos que en realidad es GetProcAddress, no sé porqué aquí fallaba, me comí mucho la cabeza sin conseguir nada, logré que funcionara pero no reparaba los saltos a partir de aquí, al final descubrí una cosa: Como tenemos que desviar dos secciones al EXE y esto lo hacemos desde VirtualAlloc se me ocurrió poner un Breakpoint Condicional Log en el RET de ésta con la siguiente condición:

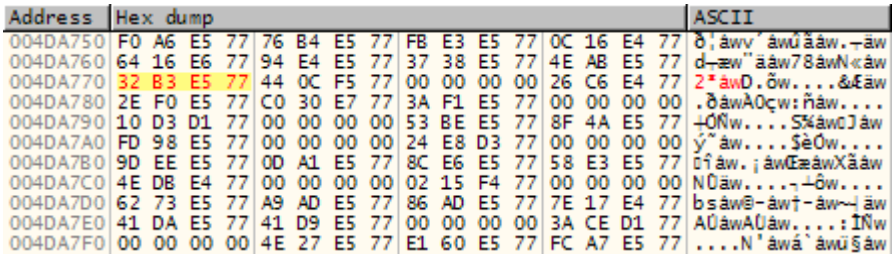


Tras unos segundos Olly para aquí con la iat y tabla de saltos correctamente:



Address	Hex dump	Disassembly	Comment
012AF08F	891F	mov [edi],ebx	
012AF091	67:64:8F06 000	pop dword ptr fs:[0]	
012AF097	65:EB 01	jmp short 012AF09B	Superfluous prefix
012AF09A	9A 83C404EB 01	call far E901:EB04C483	Far call
012AF0A1	8D7C35 61	lea edi,[ebp+esi+61]	
012AF0A5	50	push eax	
012AF0A6	52	push edx	
012AF0A7	F3:	prefix rep:	Superfluous prefix
012AF0A8	EB 02	jmp short 012AF0AC	

Tenemos la iat a partir de :004DA750 como sabemos que nos falta reparar la api GetProcAddress aprovechamos para hacerlo ahora:

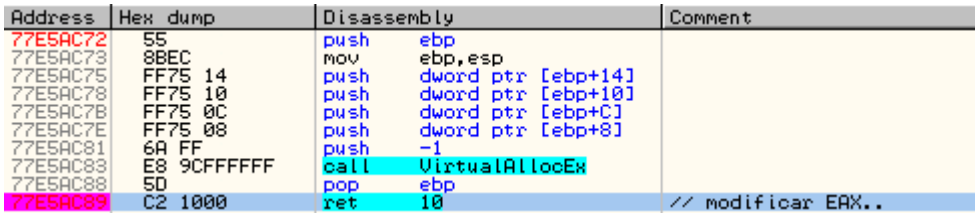


Address	Hex dump	ASCII
004DA750	F0 A6 E5 77 76 B4 E5 77 FB E3 E5 77 0C 16 E4 77	ð¦åwv´åwûãåw.¬äw
004DA760	64 16 E6 77 94 E4 E5 77 37 38 E5 77 4E AB E5 77	d¬æw"äåw78åwN«åw
004DA770	32 B3 E5 77 44 0C F5 77 00 00 00 00 26 C6 E4 77	2³åwD.õw....&Æäw
004DA780	2E F0 E5 77 C0 30 E7 77 3A F1 E5 77 00 00 00 00	.ðåwÀ0çw:ñåw....
004DA790	10 D3 D1 77 00 00 00 00 53 BE E5 77 8F 4A E5 77	¬ÓÑw....S¾åwÇJåw
004DA7A0	FD 98 E5 77 00 00 00 00 24 E8 D3 77 00 00 00 00	ý˜åw....$èÓw....
004DA7B0	9D EE E5 77 0D A1 E5 77 8C E6 E5 77 58 E3 E5 77	¥îåw.¡åwŒæåwXãåw
004DA7C0	4E DB E4 77 00 00 00 00 02 15 F4 77 00 00 00 00	NÛäw....¬┴ôw....
004DA7D0	62 73 E5 77 A9 AD E5 77 86 AD E5 77 7E 17 E4 77	bsåw©-åw†-åw~↨äw
004DA7E0	41 DA E5 77 41 D9 E5 77 00 00 00 00 3A CE D1 77	AÚåwAÙåw....:ÎÑw
004DA7F0	00 00 00 00 4E 27 E5 77 E1 60 E5 77 FC A7 E5 77	....N'åwá`åwü§åw

No nos olvidemos de modificar los dos saltos a la api:

```
00401338  - FF25 70A74D00   jmp     [4DA770]          ; kernel32.GetProcAddress
00406FDC  - FF25 70A74D00   jmp     [4DA770]          ; kernel32.GetProcAddress
```

Esto lo podríamos haber hecho después pero no importa, lo siguiente es pasar cuatro o cinco excepciones hasta llegar a VirtualAlloc:



Address	Hex dump	Disassembly	Comment
77E5AC72	55	push ebp	
77E5AC73	8BEC	mov ebp,esp	
77E5AC75	FF75 14	push dword ptr [ebp+14]	
77E5AC78	FF75 10	push dword ptr [ebp+10]	
77E5AC7B	FF75 0C	push dword ptr [ebp+C]	
77E5AC7E	FF75 08	push dword ptr [ebp+8]	
77E5AC81	6A FF	push -1	
77E5AC83	E8 9CFFFFFF	call VirtualAllocEx	
77E5AC88	5D	pop ebp	
77E5AC89	C2 1000	ret 10	// modificar EAX..

En esta primera parada tenemos EAX==0x02F00000, tras hacer algunas pruebas descubrí que esta pequeña sección de 1000h bytes la podemos desviar hacia :004DC000 donde hay un espacio libre lleno de ceros, cambiamos a EAX==0x004DC000:

```
Registers (3DNow!)
EAX 004DC000 Useful_F.004DC000
ECX 77E5AC6F kernel32.77E5AC6F
EDX 7FFE0304
EBX 0012FE90
ESP 0012FE68
EBP 0012FEA8
```

Quitamos el BPCL [BP Condicional Log] y ponemos un BP normal con F2 en el RET de la api igual y pulsamos F9, cuando vuelve a parar tenemos en EAX el valor de la segunda sección a modificar EAX==0x2F30000 que cambiamos a EAX==0x004DE000:

```
Registers (3DNow!)
EAX 004DE000 Useful_F.004DE000
ECX 77E5AC6F kernel32.77E5AC6F
EDX 7FFE0304
EBX 0012FE94
ESP 0012FE68
EBP 0012FEA8
```

Donde tenemos espacio libre para esta otra sección de 1000h bytes, y ya tenemos dos, quitamos el BP del RET de la api y llegamos al OEP como ya sabemos:

```
Address   Hex dump        Disassembly          Comment
00406C60  - FF25 70A84D00 jmp    [4DA870]      kernel32.GetModuleHandleA
00406C66    8BC0          mov    eax,eax
00406C68  - FF25 74A84D00 jmp    [4DA874]      kernel32.LocalAlloc
00406C6E    8BC0          mov    eax,eax
00406C70  - FF25 78A84D00 jmp    [4DA878]      kernel32.TlsGetValue
00406C76    8BC0          mov    eax,eax
00406C78  - FF25 7CA84D00 jmp    [4DA87C]      kernel32.TlsSetValue
00406C7E    8BC0          mov    eax,eax
00406C80    50            push   eax
00406C81    6A 40         push   40
00406C83    E8 E0FFFFFF   call   00406C68      jmp to kernel32.LocalAlloc
00406C88    C3            ret
```

Tiene mejor pinta que al principio, repasemos un poco lo que tenemos:

1.- La iat totalmente reparada
2.- La tabla de saltos arreglada también
3.- La sección [02F20000] desviada a [004DC000]
4.- La sección [02F30000] desviada a [004DE000]

Antes de continuar abrimos el Import Reconstructor v1.06f [***ImpREC***] y le damos los valores correctos acorde con la ubicación de la tabla:

Picamos en GET IMPORTS y nos saca todas las funciones, pero no haremos nada más por ahora, la salvamos como iat.txt en la carpeta de instalación del programa. Más adelante la utilizaremos. Hemos puesto el OEP en :00401000 y ya veremos el porqué más adelante. No olvidemos copiar las dos secciones de 1000h bytes al ejecutable antes de volcar el archivo.

Una vez salvada la tabla llega el momento de dumpear, lo único que debemos tener en cuenta es tener deshabilitada esta casilla:

Y en el campo OEP cambiaremos el 0x6C60h que nos pone Olly por 0x1000h que será el que utilizaremos, pulsamos DUMP y lo salvamos como "tute.exe" por decir algo, a partir de este dumpeado seguiremos el trabajo pero en la segunda parte del tute que este se ha hecho ya muy largo, bouuuufff¡¡!!!

Para cualquier cosa ya sabéis donde contactarme: syxe05@gmail.com